

3.3 - Load balancing

Distribuir a carga sem quebrar a sessão

O sistema que pede login toda hora

- **Load balancer colocado na frente da autenticação**
 - Configuração padrão: round-robin entre instâncias

LUNALOG

A LunaLog colocou um load balancer na frente do serviço de autenticação, com round-robin no padrão. A requisição de login cai na instância A, que cria a sessão. A requisição seguinte, já autenticada, cai na instância B, que nunca viu aquela sessão e responde 401. Em poucas horas, cinquenta transportadoras relatavam a mesma coisa: o sistema fica pedindo login o tempo todo. O balanceador não estava com defeito, ele apenas escancarou um design stateful que ninguém tinha enxergado antes.

Algoritmos de balanceamento

Como o tráfego é distribuído entre instâncias

- **Round-robin distribui requisições em ciclo fixo**
 - Simples e justo quando as instâncias são homogêneas
- **Least connections envia para a instância menos ocupada**
 - Melhor para requisições de duração muito variável
- **IP hash fixa o cliente na mesma instância**
 - Garante afinidade sem estado compartilhado entre nós
- **Versões ponderadas respeitam capacidade heterogênea**
 - Instância maior recebe peso maior na distribuição

Camada 4 e camada 7

Balanceamento camada 4 (transporte)

- Decide por IP e porta, sem ver o conteúdo
- Latência muito baixa e alto throughput
- Não entende rotas, headers ou cookies
- Ideal para tráfego bruto e protocolos diversos

Balanceamento camada 7 (aplicação)

- Decide lendo path, headers e cookies HTTP
- Permite roteamento por rota e por conteúdo
- Habilita sticky session via cookie
- Custo de processamento um pouco maior

Afinidade de sessão é remendo, não cura

- Sticky session prende o usuário sempre na mesma instância
- Resolve o sintoma do 401, mas não a causa raiz

Afinidade de sessão reintroduz estado na borda e quebra a elasticidade. Se aquela instância cair, a sessão do usuário cai junto. A cura real é tornar o serviço stateless.



Distribuir a carga entre instâncias ajuda muito. Mas existe um cenário em que distribuir não resolve nada: quando a mesma consulta chega quatrocentas vezes por segundo. Espalhar essa carga só significa sobrecarregar todas as instâncias ao mesmo tempo. O que falta é parar de ir buscar a mesma resposta repetidamente.

Caching distribuído: parar de repetir trabalho